```
   ___        __            ____           __
  /   | __  __/ /_____      / __ )____ ______/ /____  ______
 / /| |/ / / / __/ __ \_____/ __  / __ `/ ___/ //_/ / / / __ \
/ ___ / /_/ / /_/ /_/ /_____/ /_/ / /_/ / /__/ ,< / /_/ / /_/ /
/_/  |_\__,_/\__/\____/     /_____/\__,_/\___/_/|_|\__,_/ .___/
                                                        /_/
```

Technical Documentation

Automated GitHub Repository Backup System

Contents

1 Introduction **3**

2 Supported Platforms **3**
2.1 Supported Systems 3
2.2 Unsupported Systems 3

3 Configuration and Usage **4**
3.1 Configuration File 4
3.2 Manual Execution 4
3.3 Internal Workflow 5
3.4 Log Files 5

4 Requirements and Installation **5**
4.1 Dependencies 5
4.2 SSH Authentication Checklist 6
4.3 Role of `install.sh` 6
4.4 Required Privileges 6

5 Automated Execution **7**
5.1 Execution User 7
5.2 Backup Directory Permissions 7

6 Troubleshooting **7**
6.1 Missing Configuration File 7
6.2 `GITHUB_USERNAME` Is Not Set 8
6.3 GitHub Authentication Failure 8
6.4 Permission Denied on `BACKUP_DIR` 8

7 Security **8**

8 Known Limitations **8**

9 Conclusion **8**

1 Introduction

GitHub AutoBackup is an automated system designed to create and maintain local copies of all GitHub repositories owned by a given user. It targets a Raspberry Pi running Debian and is intended to operate in a fully headless environment, without any graphical interface.

The main objective is to preserve up-to-date local backups of remote repositories,thereby reducing the risk of data loss caused by accidental deletion, account compromise, or remote unavailability.

The solution is implemented exclusively with Bash scripts and standard command-line tools, including Git, GitHub CLI, and SSH. Execution is scheduled and managed by `systemd`, which enables unattended daily backups.

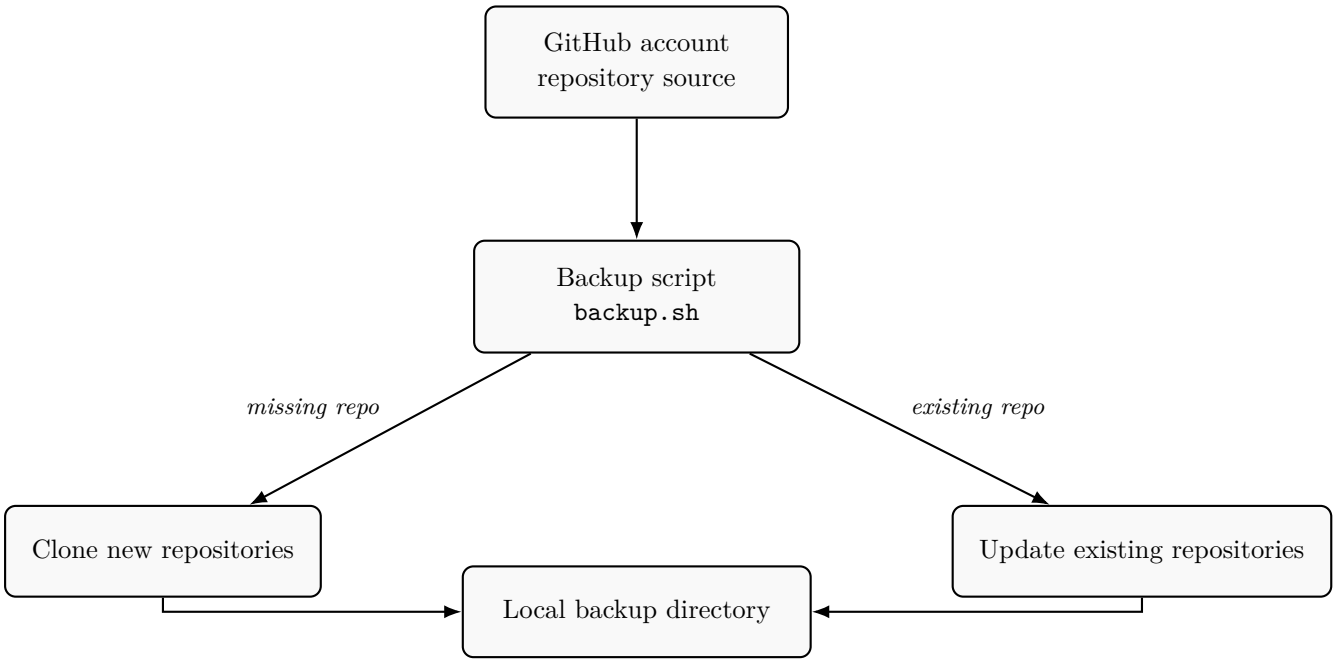

Figure 1: Global overview of the backup workflow

2 Supported Platforms

GitHub AutoBackup is designed for Linux systems using `systemd` and the `apt` package manager. The reference deployment target is a Raspberry Pi running Debian.

2.1 Supported Systems

The following environments are officially supported:

- Debian-based distributions, including Debian, Ubuntu, and Raspberry Pi OS
- `systemd` as the init system

2.2 Unsupported Systems

The `install.sh` script is not compatible with the following platforms:

- macOS
- Windows

On these systems, installation and scheduling would need to be configured manually.

3 Configuration and Usage

3.1 Configuration File

Before running GitHub AutoBackup, the configuration file located in the `config/` directory must be edited.

```
# GitHub username (without @)
GITHUB_USERNAME="Maybe1or0"

# Backup target directory
BACKUP_DIR="/home/axel/Github"

# Clone mode: normal or mirror
CLONE_MODE="normal"
```

This file defines the main runtime parameters of the backup system.
The `CLONE_MODE` option controls how repositories are stored locally:

- `normal`: repositories are cloned into `BACKUP_DIR/<repo>` and updated using `git pull -ff-only`. This mode provides a standard working copy that can also be used for development.
- `mirror`: repositories are cloned as bare mirrors into `<repo>.git` and updated using `git remote update`. This mode preserves all references, including branches and tags, but does not provide a working tree.

Note

The normal mode is suitable when local repositories may later be opened and edited. The mirror mode is better suited to pure archival and backup purposes.

3.2 Manual Execution

To run the backup process manually, execute:

```
42sh$./src/backup.sh
```

The script supports an optional `-log` flag. When enabled, both standard output and standard error are redirected to `backup.log` instead of being printed to the console.

3.3 Internal Workflow

The manual execution path follows four main stages:

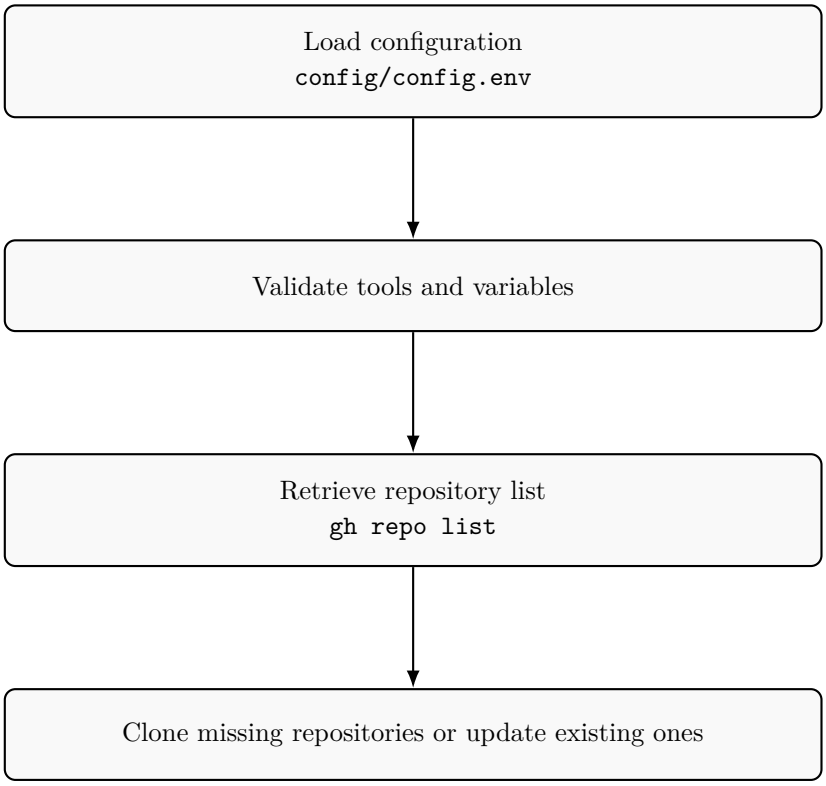

Figure 2: Internal execution flow of the backup script

3.4 Log Files

When the `-log` flag is used, logs are written to `backup.log`. A `logs/` directory is created automatically if it does not already exist.

4 Requirements and Installation

4.1 Dependencies

Each dependency serves a distinct purpose in the system:

- `git`: clone and update repositories
- `gh`: list repositories owned by the target GitHub user
- `ssh`: authenticate Git operations securely without passwords

To verify that all required dependencies are installed, run:

```
42sh$./scripts/install.sh
```

If a required program is missing, it can be installed manually with:

```
42sh$sudo apt install <program_name>
```

GitHub CLI must also be authenticated. Its status can be checked with:

```
42sh$gh auth status
```

If authentication has not yet been completed, start the interactive login process:

```
42sh$gh auth login
```

4.2 SSH Authentication Checklist

Before running the backup, SSH authentication must be functional.

- An SSH key pair exists on the system
- The public key has been added to the GitHub account
- The following command succeeds without error

```
42sh$ssh -T git@github.com
```

4.3 Role of `install.sh`

The `install.sh` script performs more than a dependency check. It also:

- verifies that required programs are installed
- copies `github-autobackup.service` and `github-autobackup.timer` into `/etc/systemd/system/`
- reloads the `systemd` configuration
- enables and starts the timer with `systemctl enable -now`

4.4 Required Privileges

The installation process modifies system-wide directories and installs `systemd` units. As a result, `install.sh` must be executed with root privileges.

Warning

The installation script should be run with `sudo` or from an account that has `sudo` privileges. Without elevated permissions, the service and timer cannot be installed correctly.

5 Automated Execution

A daily backup mode is configured through the `github-autobackup.timer` unit, which is scheduled with `OnCalendar=daily`.

To enable and start the automatic backup on a Linux system using `systemd`, run:

```
42sh$./scripts/install.sh
```

This command enables the timer and starts it immediately.

To trigger the backup service manually through `systemd`, run:

```
42sh$sudo systemctl start github-autobackup.service
```

The script may also always be executed directly:

```
42sh$./src/backup.sh
```

5.1 Execution User

The `systemd` service runs under the user account that executed `install.sh`. During installation, the username is injected dynamically into the service unit.

5.2 Backup Directory Permissions

The execution user must have write permissions on `BACKUP_DIR`. If the directory is not writable, the backup process will fail.

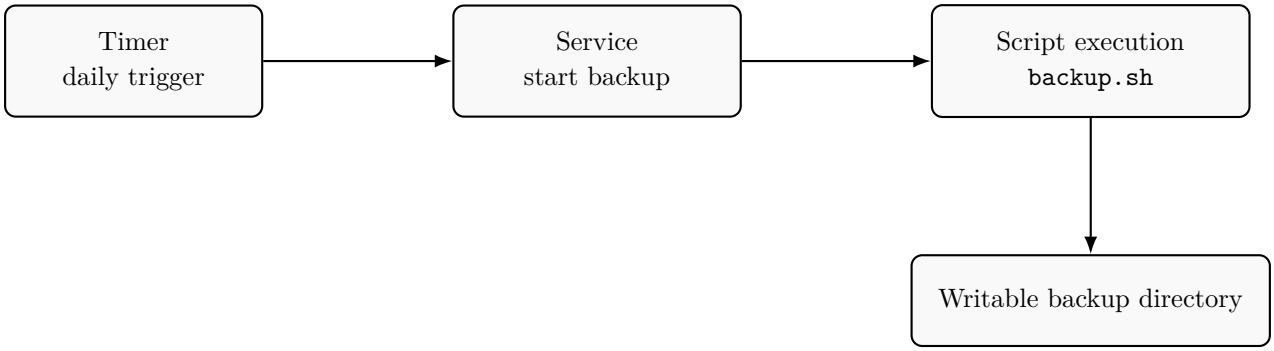

Figure 3: Automated daily execution through systemd

6 Troubleshooting

6.1 Missing Configuration File

Cause: `config.env` is missing or unreadable.

Solution: ensure that `config/config.env` exists and that the current user has permission to read it.

6.2 `GITHUB_USERNAME` Is Not Set

Cause: the variable is empty or commented out.

Solution: define `GITHUB_USERNAME` explicitly in `config.env`.

6.3 GitHub Authentication Failure

Cause: GitHub CLI is not authenticated.

Solution:

```
42sh$gh auth login
```

6.4 Permission Denied on `BACKUP_DIR`

Cause: the execution user does not have sufficient write permissions.

Solution: adjust the directory ownership or its write permissions.

7 Security

Access to GitHub repositories is performed through SSH, which allows the use of public-key authentication such as ed25519 or RSA and avoids password-based Git operations.

GitHub CLI authentication follows the principle of least privilege by relying on read-only permissions whenever possible.

Note

This project is intended for repository backup, not for repository administration. For that reason, minimal permissions should be preferred during authentication and setup.

8 Known Limitations

The repository listing is limited to 1000 repositories. This restriction comes from the internal use of the command `gh repo list -limit 1000`.

9 Conclusion

GitHub AutoBackup provides a simple and deterministic solution for maintaining local copies of GitHub repositories on a Linux system. Its design emphasizes clarity, automation, and operational reliability, making it both a practical backup utility and a clean systems scripting project.